Identifying Generated Text from an LLM is Easy. Right?

SynthIR Workshop at SIGIR 2026
J. Shane Culpepper
Professor of AI
The University of Queensland

July 24, 2026

Overview

1. Task & Collection
2. Building a Classifier
3. Results & Failure Analysis
4. Retrieval + Filtering
5. Sidebar: Engineering lessons
6. Conclusions & Future Work

Outline

The Task

GenIR Workshop Challenge 1 (SIGIR 2026): Index a "collection" containing both human and LLM-generated passages, run standard top-k retrieval per query, and recover ranking quality (MRR) by identifying/filtering generated passages out of the ranked list. Why? Because generated passages often rank highly and push human passages that are relevant down in the list.

The Task

Why Should I Care?

- Search collections will increasingly be polluted with more and more LLM-generated content.
- Generated passages compete for top-k ranking against any "true" relevant / preferred results.
- A reliable human-vs-LLM detector is the key building block for a post-retrieval filter (and most people seem to believe it is easy to identify LLM generated content).

Methodology

- Build a baseline classifier first
- Then build a state-of-the-art transformer-based one
- Test both in an actual retrieval setting, not just in isolation
- Two GPUs available (RTX 3090 + RTX 4090) for fine-tuning (my PhD students are hogging all of the other ones I have!)

The dataset

Modified MS MARCO passage collection, augmented with LLM-generated passages for a subset of queries.

File	Size
Full passage collection	8,933,787 passages
Training queries	1,596
`qrels.{train,val,test}.tsv` (verified relevant / human)	2,028 / 268 / 236
`gen.{train,val,test}.tsv` (LLM-generated)	74,419 / 10,135 / 7,410

Where Did this Collection Come From?

- I created it with the help of one of my PhD students – Zhixuan Li.
- It is derived from a research problem he is working on now.
- To create the dataset, we started with the original MSMARCO dataset.
- We then sampled 200 queries from the dev set and a few thousand from the training set.
- Then we sent the query to 5 COT-based LLM models recently released along with a detailed instruction prompt to create "pseudo-relevant" passages for each of the queries.
- Models used were: GLM-4.7-Flash-Q8, Mistral-Small-3.1-24B-Instruct, gpt-oss-20b-UD-Q8, stepfun-ai-Step-3.5-Flash-Q4, Apriel-1.5-15b-Thinker-BF16u.

The Instruction Prompt (Few Shot)

INSTRUCTIONS: Given a simple question / information need as a user prompt, generate a 50 to 300 word answer to the question that is complete and might have been created by summarizing part of a webpage from Wikipedia, Reddit, or Stack Overflow. Below are five examples of the answer for five different questions sampled from the datasat being used.

EXAMPLE 1

Question / Information Need : what is the function of the ileum
Answer : The Ileum is responsible for absorption of nutrients into the blood stream. Also the jejunum has the same function. The Ileum is the last section of the small intestine (its about 12 feet/ 3.6m in length) and joins the large intestine at the ileocecal valve. The role of the small intestine as a whole is its the major digestive system of the body.

EXAMPLE 2

Question / Information Need : an average menstrual cycle is how many days long
Answer : When to get checked out. Normal cycles last between 24 to 35 days. Some teens might have shorter cycles of only 21 days, and others might go as long as 45 days between periods. Adults can have a range of between 21 to 35 days. See a doctor if your cycle falls outside of those ranges.
.... (2 more random examples)
END INSTRUCTIONS

Outline

Model zoo, in order of sophistication

Model	Rationale
Stylometric	13 hand-crafted surface features (length, punctuation ratios, TTR, syllables/word) + LogReg — cheapest possible baseline, tests surface style alone
TF-IDF n-grams	Word(1,2) + char(3,5) TF-IDF, 150K features + LogReg — classic, strong text-classification baseline
TF-IDF + stylometric	Concatenation of the above — does combining help?
Frozen embeddings	BGE-base pretrained sentence embeddings + LogReg — tests whether *semantic* representation (not just lexical) carries a signal
LM perplexity	GPT-2-medium per-token NLL statistics (DetectGPT/GLTR-style) + LogReg — topic-agnostic *by construction*, targets fluency artifacts of generation
Fine-tuned transformer	DeBERTa-v3-large, full fine-tune, binary classification vs. pairwise/contrastive loss

Outline

First results were... disappointing

Every one of the first four model families landed on chance $= 0.50$ for our held-out queries.

Before concluding "the task is unsolvable," an audit was performed on the dataset.

The Unexpected: The original train split used `triples.train.tsv`'s `(qid, real_pid, gen_pid)` triples directly, treating `real_pid` as "the human passage for this query."

Audit: Is `real_pid` ever query-relevant?

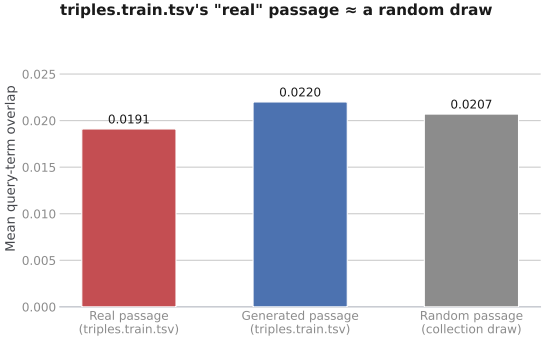
Check 1 — exact match

Does `real_pid` ever match the qrel verified relevant passage for that query?

1.36%

(1,008 / 73,919 rows)

Check 2 — What about query term overlap?

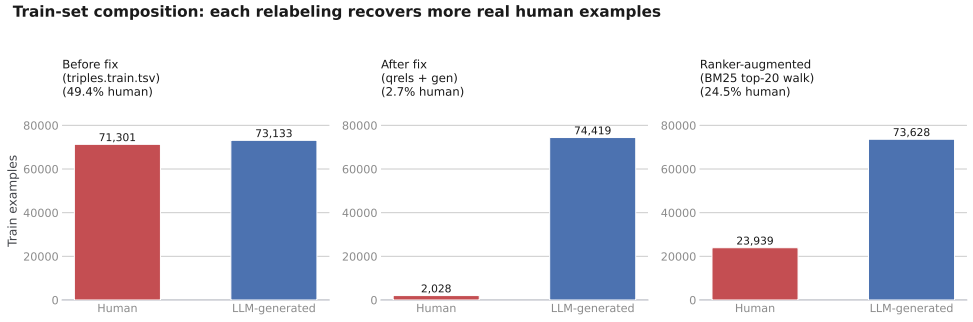


Conclusion: `triples.train.tsv`'s "real" passage is statistically indistinguishable from a random passage pulled from the whole 8.9M-passage collection — which may be OK but it is not the query-relevant one.

Why does it Matter?

Fix: Rebuild *train* the same way as val/test — `qrels.train.tsv` + `gen.train.tsv` — so every split's “human” class is population-consistent.

The fix changes the training split size and balance



From 71,301 (mostly-nonrelevant) human examples down to 2,028 genuinely query-relevant ones — smaller, but rank insensitive. Handled with class-balanced loss / oversampling. The third panel (ranker-augmented) is covered next — that ∼97–98% generated skew turned out not to be a fixed property of the collection either.

Is this skew even important?

No. The `qrels` contain only the one to three passages a human assessor judged as relevant (or more precisely – "preferred") per query (2,028 for train). But the original MS MARCO collection was built by pooling the top-100 ranked results per query — so roughly 100 human passages sit in the collection per query, not just the ones in the QREL file. The dataset README says that the `gen.{split}.tsv` lists the pids for *all* generated passages, so **any retrieved passage not in those files is guaranteed to be human**, regardless of which query surfaces it.

A Simple Recovery method: run BM25 (the same ranker used later for baseline retrieval), walk its top-100 results per query, keep every non-generated hit as an additional human example. High ranking ones yield genuine hard negatives — real, topically-relevant passages a retriever normally surfaces — unlike an approach that uses random draws, such as in the original `triples.train.tsv` file, and is not limited to the few passages in the QRELS files.

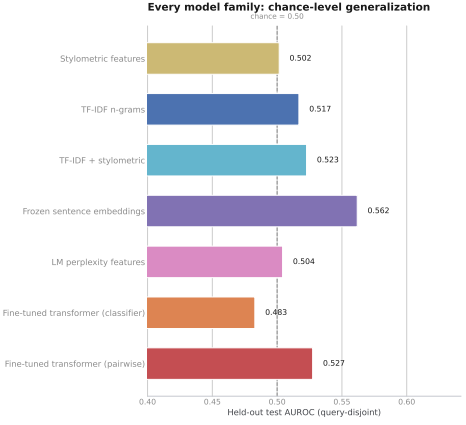
Once again, who cares?

- Recall the actual purpose of our task.
- We want to be able to filter out LLM generated passages from a top-k ranked retrieval list.
- In this scenario, we are not just randomly trying to classify passages as human or LLM.
- Why? Because any ranker worth using will only surface documents that are contextually related to a query.
- Conceptually this is much harder as now our generated LLM passages are essentially "hard negatives".
- So there are only small differences between the human and LLM generated passages.
- In theory this much harder to do, but as we will see shortly it doesn't really matter.

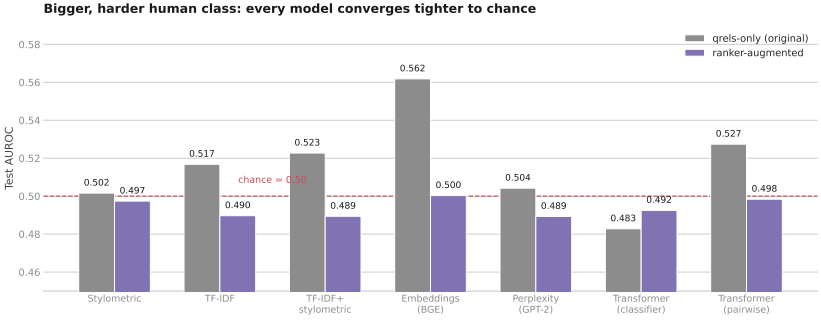
Outline

Did fixing the dataset help us? Spoiler: Nope

I re-ran the fast baselines that I created on the corrected data first; then the semantic (embeddings), fluency (perplexity), and transformer approaches, all on the new data.

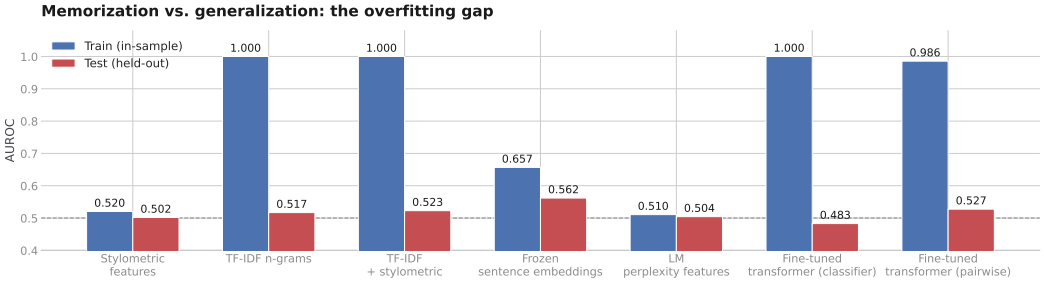


Now, an even harder test: does more (realistic) data change the verdict?



Retrained **all seven** model families — including both fine-tuned DeBERTa-v3 variants — on the ranker-augmented data (train human class: 2,028 → 23,939, real hard negatives from BM25 top-20, not qrels alone). Every model converges even tighter to chance — the null result gets *stronger*, not weaker, on a bigger and harder dataset. Embeddings' previous edge (0.562, the highest of any model) collapses to 0.500: it was apparently detecting "is this the one canonical qrels answer," not human-vs-generated authorship. Even the fine-tuned transformers, given 12× more real examples to learn from, land at 0.492 / 0.498.

Overfitting made explicit



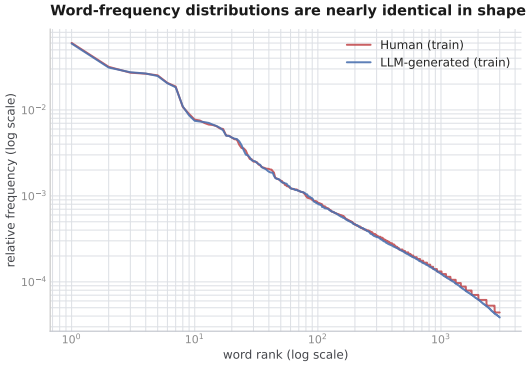
Low-capacity models (stylometric, embeddings) can't even fit train; high-capacity ones (TF-IDF, transformer) memorize train perfectly — neither generalizes.

Why doesn't this work at all?

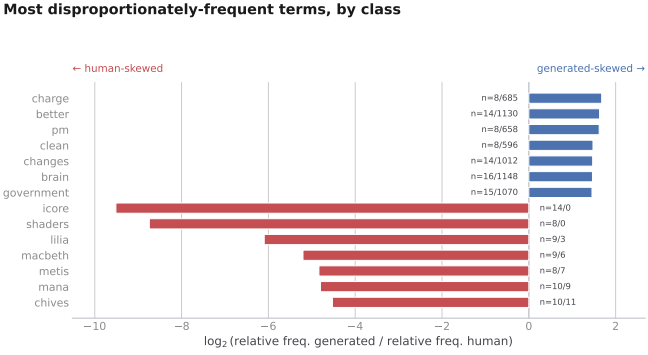
Same interrogation, at the most basic possible level: unigram frequencies, train split only (2,012 human / 73,628 generated passages, 113,561 / 4,181,565 tokens). Everything compared as *relative* frequency to control for the 37× corpus-size difference.

Statistic	Value
Jensen–Shannon divergence (0 = identical, 1 = disjoint)	0.094 bits
Zipf slope, human	−0.866
Zipf slope, generated	−0.874
Vocab size, human	15,126
Vocab size, generated	107,575

Top content words, both classes:
time, use(d), name, like, first — essentially the same top-10 list.



Surely there must be some differences in the two distributions?



Caveat visible in the counts: the human-skewed terms (`icore`, `shaders`, `macbeth`...) are single-passage artifacts (n=8–14 in a 2,012-passage corpus) — noise, not style. The generated-skewed terms (`charge`, `better`, `government`...) are statistically real but generic/topical — none of the classic "AI-tell" words (*delve, moreover, furthermore, boasts*) show up on either side.

What can we learn from this?

- The negative classification result isn't a fluke of any one model — it's visible **already at the crudest possible level of analysis**, unigram frequency.
- The two classes' vocabularies overlap almost perfectly in shape (Zipf slopes within 0.01 of each other) and in content (same top words).
- The small divergence that *does* exist doesn't look like authorial style — it looks like a sampling-depth artifact (generated passages: $\sim$46/query; human: $\sim$1.25/query, so generated naturally covers more topics more repeatedly).
- This is a genuinely different, complementary line of evidence from the modeling results — and it provides strong evidence that this is an impossibly hard task.

Full comparison: Everything I could think of

Model	Train AUC	Test AUC
Stylometric features	0.520	0.502
TF-IDF n-grams	1.000	0.517
TF-IDF + stylometric	1.000	0.523
Frozen sentence embeddings (BGE)	0.657	0.562
LM perplexity features (GPT-2)	0.511	0.504
Fine-tuned transformer (classifier)	**1.000**	**0.483**
Fine-tuned transformer (pairwise)	0.986	**0.527**
chance level = 0.50 AUC		

Every single approach lands within noise of chance on held-out queries — and all seven were independently reconfirmed on the ranker-augmented data (12× larger human class), converging even tighter to chance rather than drifting away from it.

Outline

Fine-tuning DeBERTa-v3-large: two variants

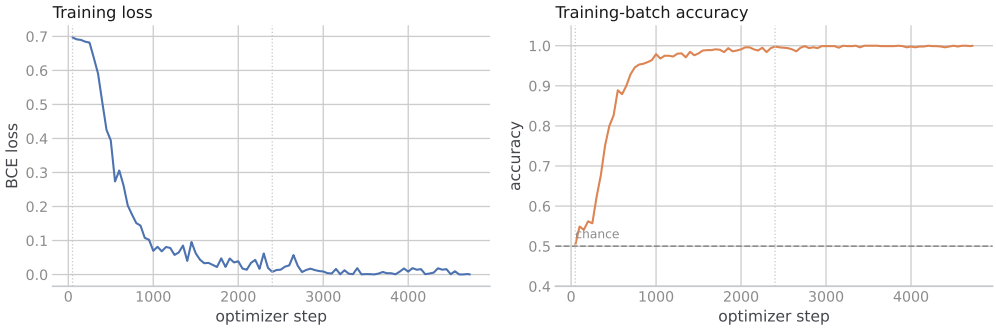
Classifier
Standard binary cross-entropy on individual labeled passages (matches the sklearn baselines' setup exactly).

Pairwise / contrastive
Margin/logistic loss directly on `(query-relevant real, generated)` pairs sharing a query — tests whether exploiting the paired structure explicitly helps the model find a query-invariant separator.

Both: mean-pooled representations, bf16 autocast, fp32 master weights, LR 2×10^{-5}, 2 epochs, on separate GPUs (classifier: RTX 4090, pairwise: RTX 3090).

The Training curves: Unreasonably rapid memorization

DeBERTa-v3-large classifier: rapid memorization



100% training-batch accuracy within one epoch. Held-out val AUC that same epoch: 0.521. Final test AUC: 0.483.

Pairwise Model Training: Much slower, same result

DeBERTa-v3-large pairwise: slower but steady fit

Final test AUC: 0.527 (train AUC 0.986 — same memorization-without-generalization pattern as every other approach).

Outline

Reframing the problem

Clarification that reshaped this investigation: the deliverable isn't a context-free "is this passage human or LLM" classifier in isolation. It is:

1. Index the full augmented collection (human + LLM-generated passages).
2. Run standard top-k retrieval per query.
3. Evaluate using the `qrels` and MRR@10.
4. Generated passages consistently compete for the top ranking positions.
5. If they are retrieved, they often push the relevant human passage down or even out of the top-k, lowering retrieval effectiveness.

The job of our classifier is to **filter out passages that are likely generated by an LLM at post-retrieval time** to recover ranking effectiveness — everything up to this point is building and stress-testing that classifier. Now we test it in the setting that actually matters.

Retrieval Setting

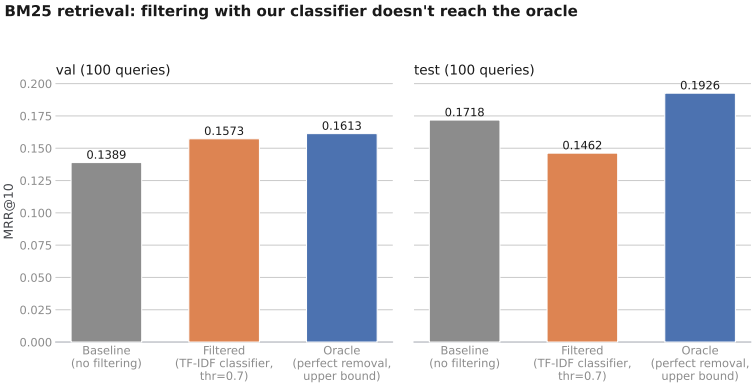
- **Index:** full collection, 8,835,396 passages, via PyTerrier / Terrier (BM25, default parameters, untuned).
- **Queries:** 100 val + 100 test queries with qrels.
- **Retrieve:** the top-100 per query.
- **Score:** MRR@10 against `qrels`.
- **Filter conditions tested:**
  - *Baseline* — no filtering.
  - *Filtered* — drop passages using a TF-IDF+LogReg when the score is above a generated-probability threshold.
  - *Oracle* — drop passages using the true `gen.{split}.tsv` labels to establish the upper bound (This is cheating and not achievable in deployment) but it is good to know what is possible.

Surprise! Generated passages are getting retrieved

Split	Mean % of top-100 generated	Relevant found in top-100	Median rank of first hit
val	12.5%	100%	9
test	15.4%	100%	14

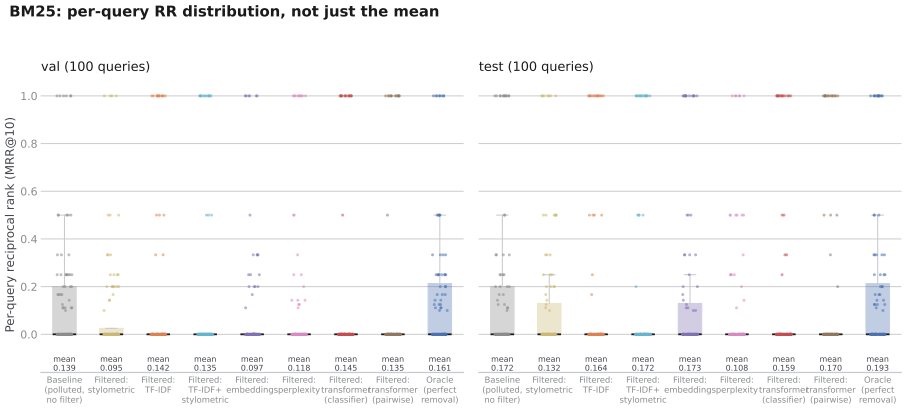
Note this is much lower than the 97–98% figure from the classification datasets earlier — that figure was an artifact of how those datasets were constructed (all known generated passages vs. only the few known-relevant ones), not a property of a real retrieved list. Queries don't touch most of the 8.8M-passage collection, so most of a real top-k list is ordinary, unrelated real content — but a non-trivial 12–15% *is* LLM-generated competing for ranking slots.

Does filtering help? In theory, yes. In practice, it's complicated.



Oracle (perfect knowledge) improves MRR by +12% to +16% relative — confirms the effort is worth it *in principle*. Our actual classifier's effect on aggregate MRR is inconsistent: hurts on test (−0.026), slightly helps on val (+0.018) at threshold 0.7 — looks like noise around zero, not a useful result.

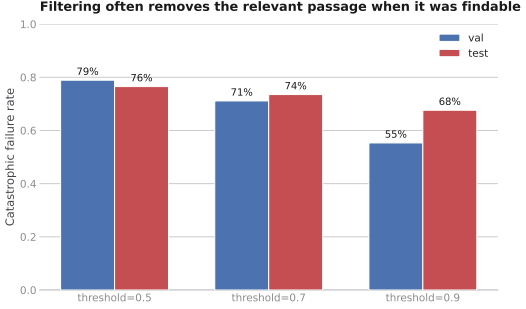
The entire per-query picture – arithmetic means tell us nothing



Every box is dominated by zero (relevant passage outside top-10) — means alone hide this. TF-IDF-filtered boxes barely move from baseline (no real skill, as expected). Stylometric-filtered is visibly worse (fewer high-RR points) — an unskilled filter actively destroys value here. Oracle visibly shifts upward. This view is BM25-only; the dense-retriever comparison follows shortly.

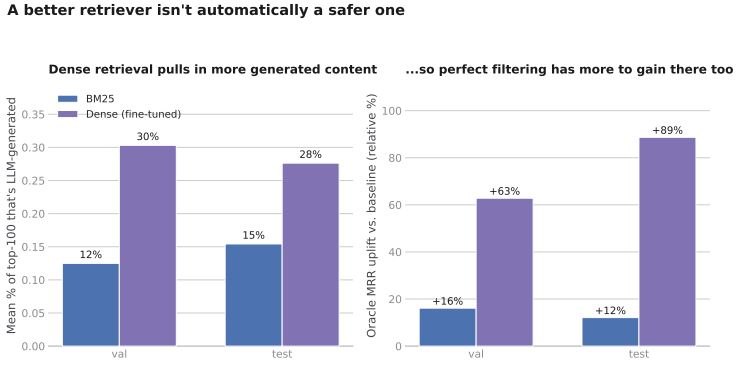
The Worst Case Scenario

The real risk isn't degrading the effectiveness — it is a *false positive on our relevant passage*. If the filter incorrectly flags the one true relevant passage as "generated," the MRR for that query drops to **exactly zero** — which is worse than doing nothing at all.



Even at the most conservative threshold tested, our classifier destroys a previously-findable relevant result more than half the time — the direct, practical consequence of the chance-level AUC established throughout this investigation.

Does a ranking model fix this? Spoiler: Nope — it can even make it worse



Dense retriever: BAAI/llm-embedder, fine-tuned on this collection's own `qrels.train.tsv` — off-the-shelf MS MARCO retrievers are invalid here since the collection has been modified. A stronger retriever finds more of everything, including generated passages — top-100 contamination roughly doubles (12–15% → 28–30%), so the oracle upside (and the exposure without a safe filter) grows, not shrinks.

So What Have We Actually Learned Here?

- Framing this task as retrieval confirms the underlying problem is real and worth solving: Generated passages compete for top-k rank positions, and perfect filtering would meaningfully improve effectiveness — for every model tested.
- But it sharpens the bar for "good enough to deploy": aggregate MRR improvements are not sufficient evidence of a working filter — a classifier can look aggregate-neutral or positive while catastrophically failing for individual queries most of the time (dense retrieval catastrophic rate is 55–79%, essentially matching BM25).
- None of the classifiers built here (chance-level AUC throughout) are safe to deploy as a retrieval filter given such a high failure rate with *either* retrieval model.
- **Upgrading the retrieval model is not a substitute for a working filter** — it changes how much contamination there is, but not whether our classifier can safely remove it.

Outline

Lessons learned during these experiments

Stage	Lesson
Label derivation	The primary use case of the dataset is ranking, not classification — so labels must be carefully derived. Never forget the end goal. Classification is not orthogonal to retrieval.
Evaluation design	Query-disjoint splits are necessary but not sufficient — always cross-check with pooled GroupKFold and perform population audits (e.g. query-term overlap) before spending hours training a model.
Baseline modeling	In-sample fit (even AUC $\approx$ 1.0) tells you nothing about generalization when features vastly outnumber minority-class examples — the train/test *gap* is an important signal that something is terribly wrong.
Transformer fine-tuning	A pretrained checkpoint that declares `torch_dtype` can silently mismatch with an autocast dtype and produce NaNs with no error — So force fp32 master weights explicitly when in doubt. Diagnosing if a model "is it learning" needs *two* sanity checks: A tiny-fixed-batch overfitting test (to ensure pipeline correctness) and a real diversity-batch test (aka task learnability).
Using a new Collection	When you decide to use a new test collection, make sure you understand how it was created. Otherwise, you are in real trouble if it does not work as it should. Fine tuning big models is costly. Be sure that you trust the data before you commit your time and resources. Our negative result got *stronger* as I increased model complexity using **seven** different models, including both fine-tuned transformers.
Scaling pairwise training	Re-deriving *pairwise*/contrastive training pairs from a huge label pool doesn't scale naively — the full real$\times$gen cross product would have been 10$\times$ larger ($\sim$19h instead of $\sim$40min); sampling one real per generated passage kept the pair count (and runtime) in the original ballpark. (i.e. Carelessness on my part would have led to an experiment that would still be running while I am giving this talk!)

Outline

So, where does this leaves us?

- **Seven** structurally different models (including two fine-tuned state-of-the-art transformers), **three** independently constructed training sets (buggy triples, qrels-only, ranker-augmented), **one** consistent result: chance-level generalization to unseen queries is the best you will ever get.
- This is a surprising finding to me about the dataset and the task and not an artifact of any single model pipeline. It is not an artifact of an undersized or unrepresentative human class either: a $12\times$ larger, hard-negative-enriched human class made every model converge *even tighter* to chance, not less — including the fine-tuned transformers, which had the most capacity to exploit any signal if one existed.
- In-sample overfitting is trivially easy (every high-capacity model reaches AUC ≈ 1.0 on train). So clearly we are training the models correctly – the difficulty is entirely in trying to generalize the model that is learned.

Future Work

1. **Investigate generation-technique heterogeneity.** The collection used a variety of generation techniques per query. Clustering the generated passages by LLM generator might reveal that some subgroups are far more detectable than others, even though the aggregate converges to what looks like normal human text.
2. **Align classifier model with LLM generator.** Building the classifier using the same model that generated the text isn't realistic in deployment, but trying it would confirm whether this is harder than we thought, or whether alignment makes the task tractable.
3. **Reconsider the task framing.** Perhaps we are trying to solve a problem we are not yet ready to solve — contextually similar passages make this genuinely hard. Worth stepping back to check whether human/LLM text is even separable i.i.d.
4. **Live with a negative result and learn from it.** I find this result both surprising and informative — and worth reporting. A rigorously verified result using seven model families and three data constructions, with a full engineering audit trail, is now documented for future work with a similar goal.
5. What to have a go? https://github.com/jsc/genIR.


That's all Folks!
BUGS BUNNY